



Evolutionary Computation

Assignment 3: Local Search

BENGRICHE Tidiane (ER-2037)
SZCZUBLIŃSKA Joanna Wiktoria (156070)



Contents

1	Problem	1
2	Pseudocode	2
3	Results	7
4	Conclusion	11

1 Problem

The goal of this project is to implement and analyze **local search algorithms** for a combinatorial optimization problem. The local search improves an initial feasible solution iteratively by exploring its neighborhood and applying moves that reduce the objective function value.

We implement two main variants of local search:

- **Steepest Descent (Steepest) Local Search**
- **Greedy (First-Improvement) Local Search**

Additionally, we investigate different **neighborhood structures** and **starting solution strategies**, leading to eight algorithmic variants in total.

The experiment combines three binary design choices:

Component	Option 1	Option 2
Type of Local Search	Steepest Descent	Greedy
Type of Intra-Route Moves	Node Exchange	Edge Exchange
Type of Starting Solution	Random	Greedy Cycle

The purpose of this report is to present the implementation of these methods in C++ and to see how these new features improve the results from last week.

You can see the repository of our assignment on github:

https://github.com/tbengric/EC_laboratory

2 Pseudocode

In this section, we will present the pseudocodes of the algorithms. For this assignment, we will code in C++.

Cost Change for Swapping Two Nodes in the Same Path

```
FUNCTION delta_intra_node(path, first_node_index,
second_node_index, dist_matrix):
    path_size ← LENGTH(path)
    IF path_size < 2 THEN :
        RETURN 0

    prev1 ← path[(first_node_index - 1 + path_size) % path_size]
    node1 ← path[first_node_index]
    next1 ← path[(first_node_index + 1) % path_size]

    prev2 ← path[(second_node_index - 1 + path_size) % path_size]
    node2 ← path[second_node_index]
    next2 ← path[(second_node_index + 1) % path_size]

    old_cost ← 0
    new_cost ← 0

    IF (index1 + 1) % path_size = index2 THEN :
        old_cost ← dist_matrix[prev1][node1] + dist_matrix[node1][node2] +
dist_matrix[node2][next2]
        new_cost ← dist_matrix[prev1][node2] + dist_matrix[node2][node1] +
dist_matrix[node1][next2]

    ELSE IF (index2 + 1) % path_size = index1 THEN :
        old_cost ← dist_matrix[prev2][node2] + dist_matrix[node2][node1] +
dist_matrix[node1][next1]
        new_cost ← dist_matrix[prev2][node1] + dist_matrix[node1][node2] +
dist_matrix[node2][next1]

    ELSE :
        old_cost ← dist_matrix[prev1][node1] + dist_matrix[node1][next1] +
dist_matrix[prev2][node2] + dist_matrix[node2][next2]
        new_cost ← dist_matrix[prev1][node2] + dist_matrix[node2][next1] +
dist_matrix[prev2][node1] + dist_matrix[node1][next2]

    RETURN new_cost - old_cost
```

Cost Change for Edge Swap

```
FUNCTION delta_intra_edge(path, first_node_of_first_edge,
first_node_of_second_edge, dist_matrix):
  path_size ← LENGTH(path)
  IF path_size < 4 THEN :
    RETURN 0

  nodeA ← path[first_node_of_first_edge]
  nodeB ← path[(first_node_of_first_edge + 1) % path_size]

  nodeC ← path[first_node_of_second_edge]
  nodeD ← path[(first_node_of_second_edge + 1) % path_size]

  IF nodeB = nodeC THEN :
    RETURN 0

  old_cost ← dist_matrix[nodeA][nodeB] + dist_matrix[nodeC][nodeD]
  new_cost ← dist_matrix[nodeA][nodeC] + dist_matrix[nodeB][nodeD]

  RETURN new_cost - old_cost
```

Cost Change for Replacing a Node

```
FUNCTION delta_inter_node(path, selected_node_index,
not_selected_node, dist_matrix, nodes):
  path_size ← LENGTH(path)
  IF path_size < 2 THEN :
    RETURN 0

  node_id_in ← path[selected_node_index]
  prev_node ← path[(node_index_in - 1 + path_size) % path_size]
  next_node ← path[(node_index_in + 1) % path_size]

  old_cost ← dist_matrix[prev_node][node_id_in] +
dist_matrix[node_id_in][next_node] + nodes_info[node_id_in].cost
  new_cost ← dist_matrix[prev_node][not_selected_node] +
dist_matrix[not_selected_node][next_node] +
nodes_info[not_selected_node].cost

  RETURN new_cost - old_cost
```

Iterative Improvement by Best Move

```
FUNCTION steepest_local_search(initial_path, dist_matrix,
intra_method, nodes):
  current_path ← COPY(initial_path)

  REPEAT :
    best_delta ← 0
    best_move ← NULL
    path_size ← LENGTH(current_path)
    IF path_size < 2 THEN :
      BREAK

    (selected, not_selected) ← find_selected_nodes(current_path, nodes_info)

    IF intra_method = "node" THEN :
      FOR i FROM 1 TO path_size - 2 DO :
        FOR j FROM i + 1 TO path_size - 2 DO :
          delta ← delta_intra_node(current_path, i, j, dist_matrix)
          IF delta < best_delta THEN :
            best_delta ← delta
            best_move ← (type: 0, p1: i, p2: j)
    ELSE IF intra_method = "edge" THEN :
      FOR i FROM 0 TO path_size - 2 DO :
        FOR j FROM i + 2 TO path_size - 2 DO :
          delta ← delta_intra_edge(current_path, i, j, dist_matrix)
          IF delta < best_delta THEN :
            best_delta ← delta
            best_move ← (type: 1, p1: i, p2: j)

    FOR i FROM 1 TO path_size - 2 DO :
      FOR node_id_out IN not_selected DO :
        delta ← delta_inter_node(current_path, i, node_id_out, dist_matrix,
nodes_info)
        IF delta < best_delta THEN :
          best_delta ← delta
          best_move ← (type: 2, p1: i, p3: node_id_out)

    IF best_move ≠ NULL AND best_delta < 0.0 THEN :
      IF best_move.type = 0 THEN :
        SWAP(current_path[best_move.p1], current_path[best_move.p2])
      ELSE IF best_move.type = 1 THEN :
        REVERSE(current_path, from: best_move.p1 + 1, to: best_move.p2)
      ELSE IF best_move.type = 2 THEN :
        current_path[best_move.p1] ← best_move.p3
    ELSE :
      BREAK

  RETURN current_path
```

Iterative Improvement by First Move Found (Part One)

```
FUNCTION greedy_local_search(initial_path, dist_matrix, intra_method,
nodes):
```

```
    current_path ← COPY(initial_path)
```

```
    REPEAT :
```

```
        path_size ← LENGTH(current_path)
```

```
        IF path_size < 2 THEN :
```

```
            BREAK
```

```
        (selected, not_selected) ← find_selected_nodes(current_path, nodes_info)
```

```
        possible_moves ← []
```

```
        IF intra_method = "node" THEN :
```

```
            FOR i FROM 1 TO path_size - 2 DO :
```

```
                FOR j FROM i + 1 TO path_size - 2 DO :
```

```
                    APPEND (type: 0, p1: i, p2: j, p3: -1) TO possible_moves
```

```
        ELSE IF intra_method = "edge" THEN :
```

```
            FOR i FROM 0 TO path_size - 2 DO :
```

```
                FOR j FROM i + 2 TO path_size - 2 DO :
```

```
                    APPEND (type: 1, p1: i, p2: j, p3: -1) TO possible_moves
```

```
        FOR i FROM 0 TO path_size - 1 DO :
```

```
            FOR node_id_out IN not_selected DO :
```

```
                APPEND (type: 2, p1: i, p2: -1, p3: node_id_out) TO possible_moves
```

```
        IF IS_EMPTY(possible_moves) THEN :
```

```
            BREAK
```

```
        SHUFFLE(possible_moves)
```

```
        found_improvement ← false
```

Iterative Improvement by First Move Found (Part Two)

```
FOR move IN possible_moves DO :
    delta ← 0
    IF move.type = 0 THEN :
        delta ← delta_intra_node(current_path, move.p1, move.p2,
dist_matrix)
    ELSE IF move.type = 1 THEN :
        delta ← delta_intra_edge(current_path, move.p1, move.p2,
dist_matrix)
    ELSE IF move.type = 2 THEN :
        delta ← delta_inter_node(current_path, move.p1, move.p3,
dist_matrix, nodes_info)

    IF delta < 0.0 THEN :
        IF move.type = 0 THEN :
            SWAP(current_path[move.p1], current_path[move.p2])
        ELSE IF move.type = 1 THEN :
            REVERSE(current_path, from: move.p1 + 1, to: move.p2)
        ELSE IF move.type = 2 THEN :
            current_path[move.p1] ← move.p3

        found_improvement ← true
        BREAK

    IF NOT found_improvement THEN :
        BREAK

RETURN current_path
```

In the greedy local search, the order of move evaluation is randomized at the start of each iteration using the C++ Standard Library function `std::shuffle`, which employs a Mersenne Twister pseudo-random number generator seeded by `std::random_device`. This randomization ensures that each iteration explores a different random subset and order of moves. The algorithm stops evaluating moves as soon as it finds an improving one, which minimizes the number of evaluated moves and speeds up convergence compared to the steepest approach.

3 Results

After trying these different algorithms, we will check their effectiveness by showing the constructed cycle.

In the plot, the cost of each node is represented by its size: larger nodes indicate a higher cost, while smaller ones indicate a lower cost. Additionally, all possible nodes are displayed, but the nodes chosen for the path are shown in blue, with their IDs printed on them. The started node is pointed with red color.

Firstly, we will show the figures for the previous algorithms :

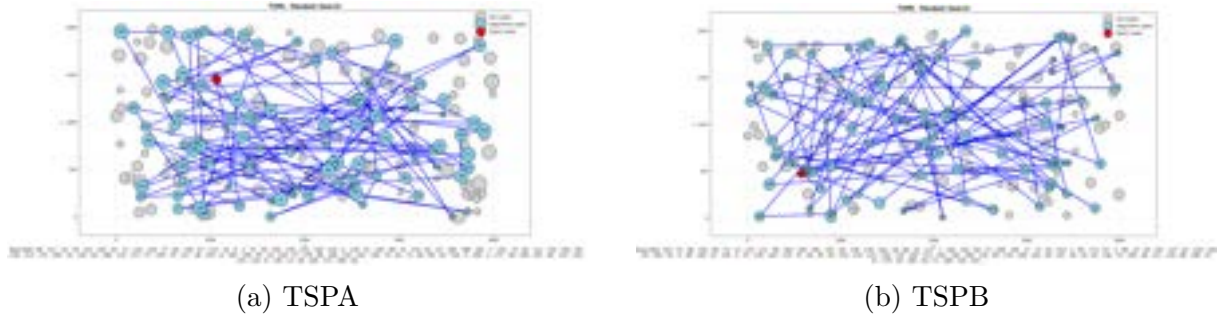


Figure 1: Random Search

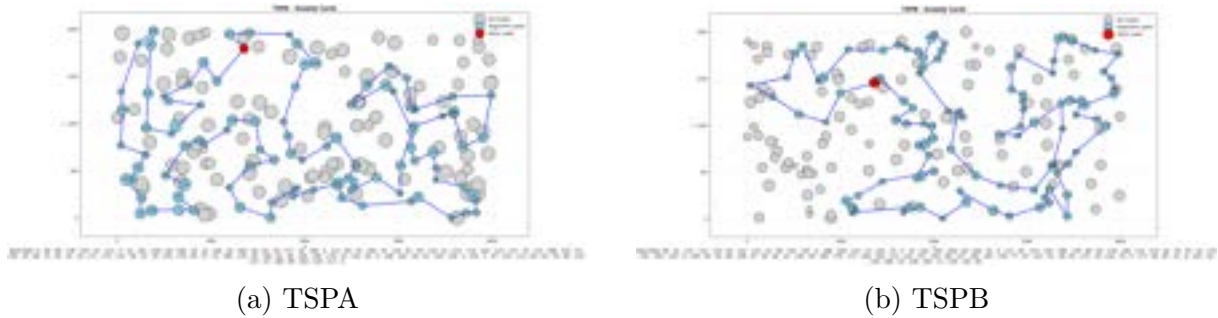


Figure 2: Greedy Cycle

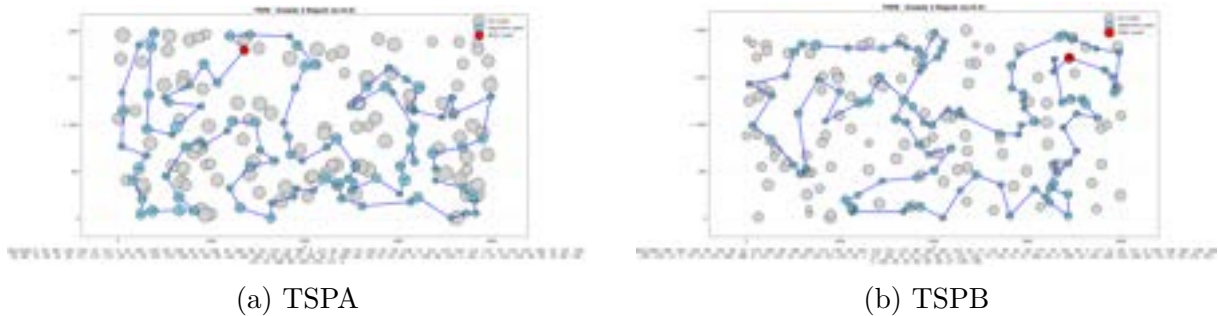


Figure 3: Greedy 2-Regret ($w=0.5$)

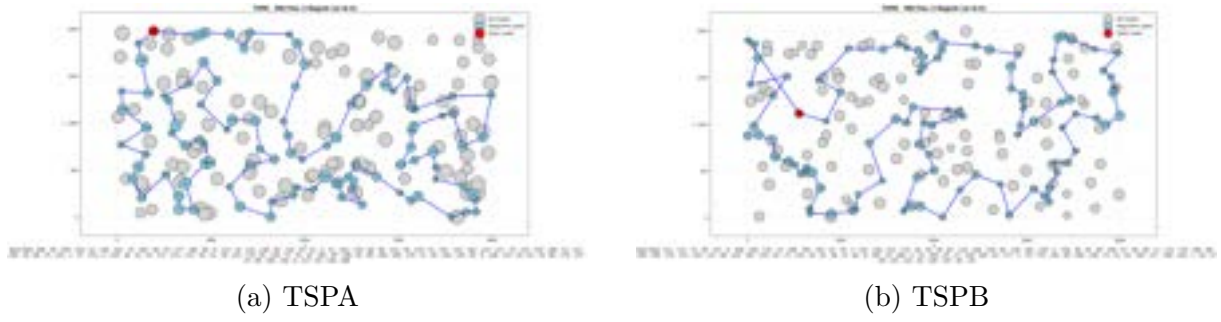
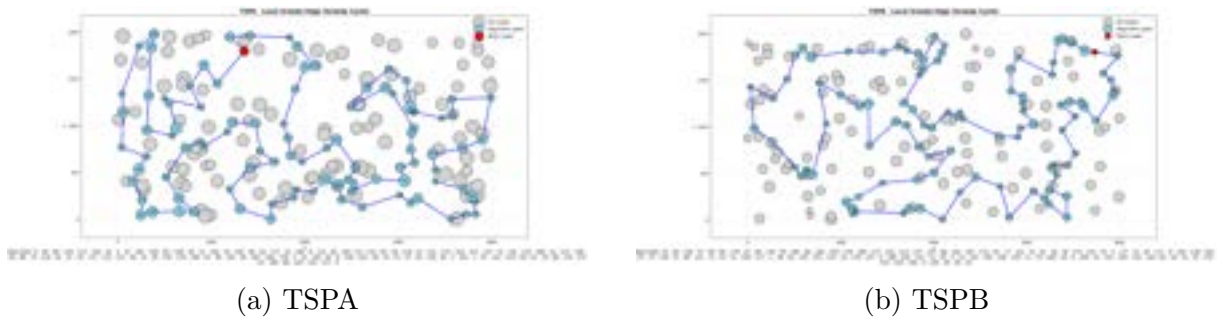

Figure 4: NN-Flex 2-Regret ($w=0.5$)


Figure 5: Local Greedy Edge (Greedy Cycle)

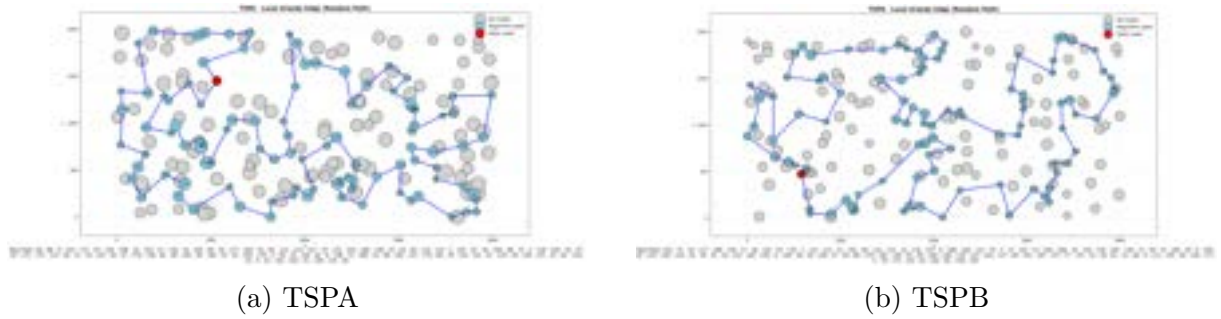


Figure 6: Local Greedy Edge (Random Path)

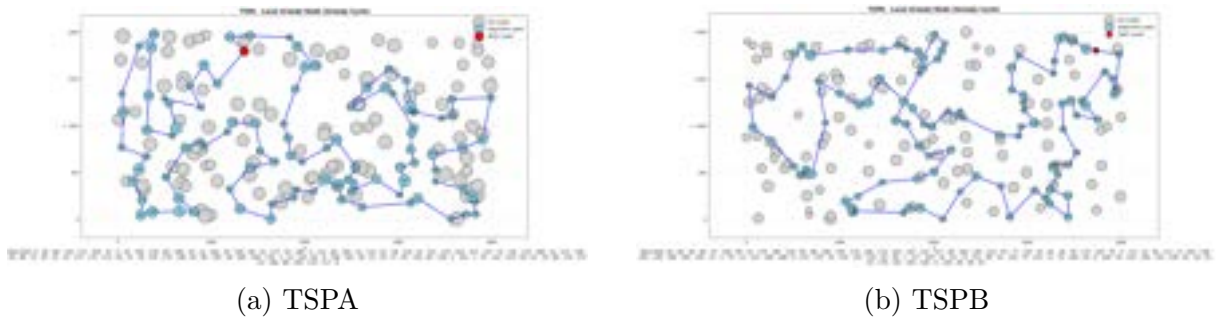


Figure 7: Local Greedy Node (Greedy Cycle)

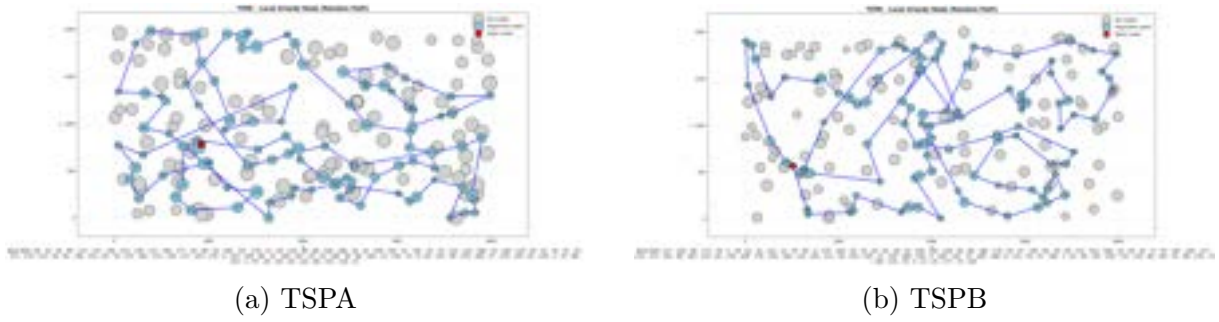


Figure 8: Local Greedy Node (Random Path)

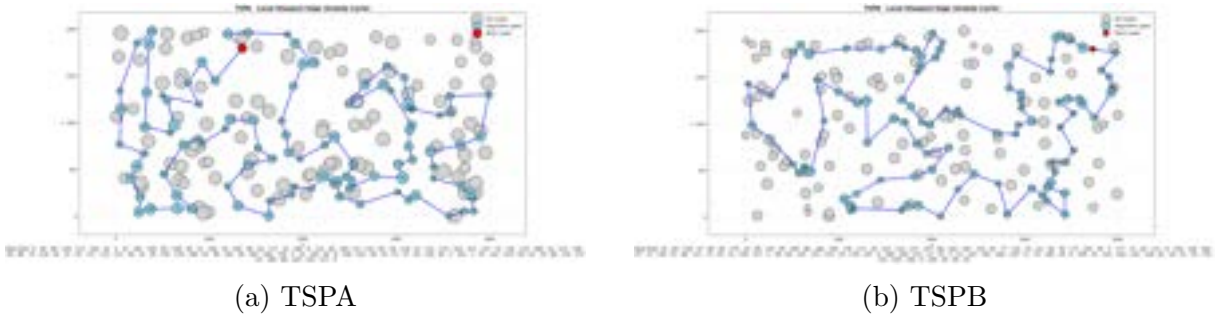


Figure 9: Local Steepest Edge (Greedy Cycle)

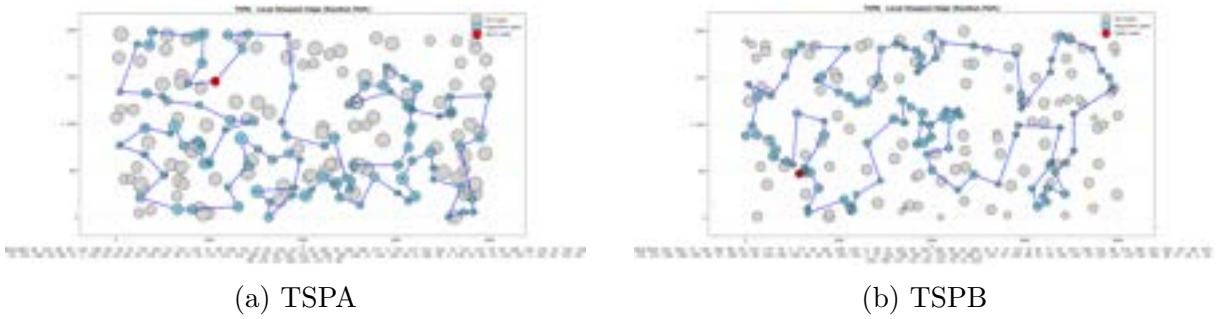


Figure 10: Local Steepest Edge (Random Path)

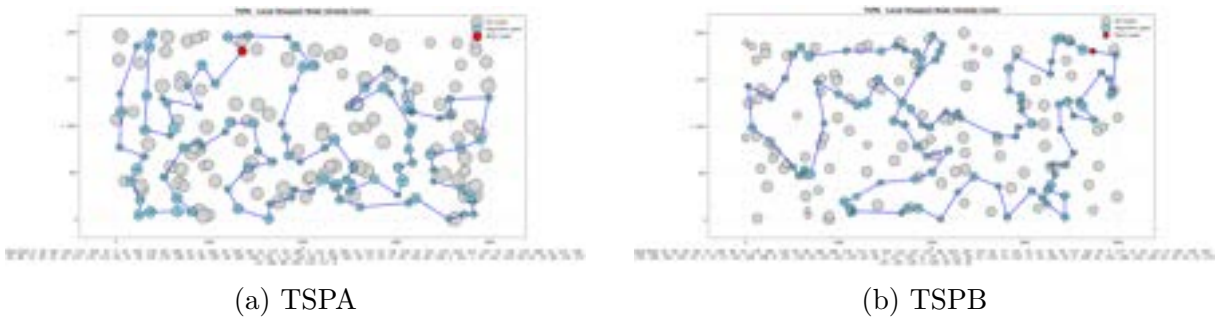


Figure 11: Local Steepest Node (Greedy Cycle)

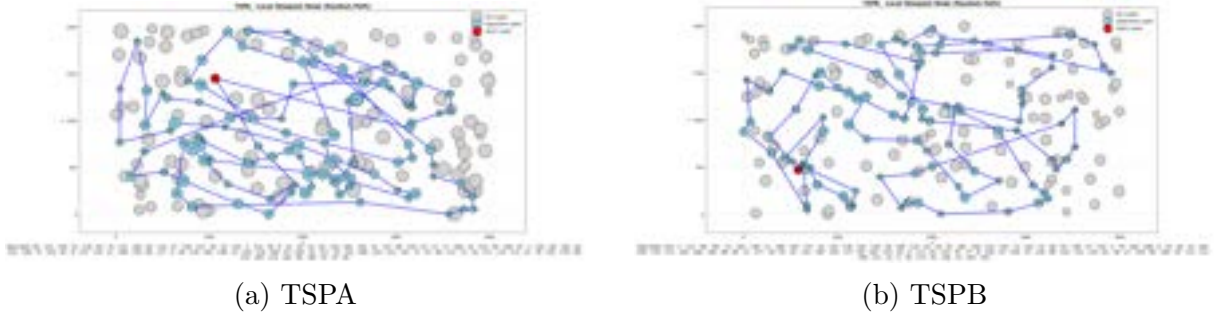


Figure 12: Local Steepest Node (Random Path)

Method	TSPA Avg (Min, Max)	TSPB Avg (Min, Max)
Random Path	263673.93 (240120, 291281)	214748.49 (189297, 236642)
Nearest Neighbor (End)	85108.51 (83182, 89433)	54390.43 (52319, 59030)
Nearest Neighbor (Flexible)	73594.10 (71868, 75953)	48694.25 (44609, 57283)
Greedy Cycle	72635.98 (71488, 74410)	51400.60 (49001, 57324)
Greedy 2-Regret	115400.24 (105692, 126860)	72712.85 (67809, 78406)
Greedy 2-Regret (w=0.5)	72136.15 (71108, 73395)	50934.60 (47144, 55700)
NN-Flex 2-Regret	117138.49 (108151, 124921)	74444.46 (69933, 80278)
NN-Flex 2-Regret (w=0.5)	72407.59 (70010, 75452)	47664.46 (44891, 55247)
Local Greedy Node (Greedy Cycle*)	70687.00 (70687, 70687)	46440.45 (46328, 51512)
Local Greedy Edge (Greedy Cycle*)	70615.62 (70571, 70663)	46085.43 (45824, 51345)
Local Steepest Node (Greedy Cycle*)	70687.00 (70687, 70687)	46466.54 (46371, 51512)
Local Steepest Edge (Greedy Cycle*)	70571.00 (70571, 70571)	45996.84 (45867, 51259)
Local Greedy Node (Random Path)	86849.33 (79842, 95043)	61779.23 (54635, 70183)
Local Greedy Edge (Random Path)	73758.43 (70930, 77926)	48761.83 (45899, 52106)
Local Steepest Node (Random Path)	96152.00 (96152, 96152)	60650.00 (60650, 60650)
Local Steepest Edge (Random Path)	74482.00 (74482, 74482)	49960.00 (49960, 49960)

Table 1: Comparison of average, minimum, and maximum objective values for TSPA and TSPB. The 3 lowest average values in each column are highlighted in bold. Greedy Cycle* is the Greedy Cycle 2-Regret (w=0.5)

Method	TSPA Time Avg (Min, Max)	TSPB Time Avg (Min, Max)
Random Path	0.0055 (0.0045, 0.0274)	0.0056 (0.0048, 0.0643)
Nearest Neighbor (End)	0.0912 (0.0555, 0.2948)	0.0905 (0.0560, 0.2582)
Nearest Neighbor (Flexible)	119.4480 (108.6357, 163.7095)	121.4057 (108.1165, 207.9213)
Greedy Cycle	114.8849 (103.9431, 197.6271)	116.5195 (105.2606, 170.2585)
Greedy 2-Regret	16.9404 (15.2202, 25.5470)	16.9604 (15.2507, 22.8133)
Greedy 2-Regret (w=0.5)	16.2875 (14.8017, 23.7737)	15.9777 (13.7575, 25.5899)
NN-Flex 2-Regret	17.5584 (15.7563, 25.9890)	17.6004 (15.7682, 30.3454)
NN-Flex 2-Regret (w=0.5)	17.2790 (15.3957, 24.9158)	17.1715 (15.4563, 28.0041)
Local Greedy Node (Greedy Cycle*)	1.6790 (1.0874, 3.5032)	2.5908 (1.4364, 4.0538)
Local Greedy Edge (Greedy Cycle*)	2.0884 (1.3517, 5.0434)	3.5161 (2.1036, 6.7556)
Local Steepest Node (Greedy Cycle*)	0.7063 (0.5997, 1.2920)	1.0081 (0.5974, 2.0411)
Local Steepest Edge (Greedy Cycle*)	0.7187 (0.6199, 1.4059)	1.3677 (0.8459, 2.3887)
Local Greedy Node (Random Path)	79.7302 (61.9555, 178.1138)	76.6896 (62.2615, 109.3885)
Local Greedy Edge (Random Path)	78.5840 (61.1249, 167.9048)	75.2933 (61.3513, 104.2782)
Local Steepest Node (Random Path)	14.9098 (13.8184, 25.5049)	17.7184 (15.4158, 58.6154)
Local Steepest Edge (Random Path)	11.4248 (10.5233, 28.2868)	11.8425 (10.2978, 26.8849)

Table 2: Comparison of average, minimum, and maximum execution times in microseconds for TSPA and TSPB. Greedy Cycle* is the Greedy Cycle 2-Regret (w=0.5)

4 Conclusion

Based on the results in Tables 1 and 2, the following observations can be made:

- **Edge vs Node Local Search:** Edge-based local search methods generally slightly outperform node-based methods in both TSPA and TSPB. This suggests that modifying edges tends to produce more impactful improvements on tour length than moving individual nodes, although the difference is not huge.
- **Steepest vs Greedy Local Search:** Local Steepest methods tend to perform slightly faster than Local Greedy in many cases, especially for edge-based improvements. While differences are not dramatic, using steepest descent often leads to marginally slightly reduced computation times but using Local Greedy tends to better objective values.
- **Impact of Initial Solution:** Starting local search from a *Greedy Cycle* leads to only minor improvements compared to starting from a *Random Path*. This suggests that a good initial solution already captures most of the problem structure, leaving less space for further enhancement.
- **Trade-off Between Quality and Time:** If the absolute best solution is required, local search provides the lowest objective values, but at a higher computational cost. Conversely, if faster results are preferred, edge-based local search starting from a *Random Path* gives reasonably good solutions efficiently.
- **General Recommendation:** Overall, the choice of method should depend on the priority: weighted 2-Regret ($w = 0.5$) is preferable for quick and efficient solutions, while steepest edge-based local search for weighted 2-Regret ($w = 0.5$) Greedy Cycle is recommended for achieving the absolute lowest objective values.